HOMEWORK 2

GitHub Link

1. GAN (15 Points)

(a) Typical Network Architecture and Training Loss

A **Generative Adversarial Network (GAN)** consists of two neural networks trained in opposition:

Generator (G)

- Takes random noise z \sim p(z) as input.
- Produces synthetic samples G(z) intended to resemble real data.

Discriminator (D)

- Takes a sample x and outputs the probability that the sample is real rather than generated.

The two models are trained through a **minimax objective**:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_D(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$$

The discriminator learns to correctly classify real and generated samples, while the generator learns to fool the discriminator.

(b) Equivalent Loss Between Model and Data Distribution

When the discriminator is optimal, the GAN objective corresponds to minimizing the **Jensen–Shannon (JS) divergence** between:

- The **data distribution** $p_D(x)$
- The **model distribution** $p_{THETA}(x)$

Thus, GAN training attempts to minimize:

$$JS(p_D(x) \parallel p_\theta(x))$$

The ideal solution occurs when the generated distribution matches the true data distribution.

(c) Potential Issues and Mitigation

Issue

The **JS divergence can lead to vanishing gradients** when the generated distribution and real data distribution have little overlap.
In such cases the discriminator easily separates real and fake samples, providing very weak gradient signals to the generator. This can result in unstable training or **mode collapse**.

Mitigation Approaches

Common solutions include:

- **Wasserstein GAN (WGAN)** — replaces JS divergence with Wasserstein distance to provide smoother gradients.
- **Gradient Penalty (WGAN-GP)** — stabilizes WGAN training by enforcing the Lipschitz constraint.
- **Non-saturating generator loss** — maximizing \log D(G(z)) instead of minimizing \log(1 - D(G(z))) for stronger gradients.
- **Training stabilization techniques** such as label smoothing or balancing discriminator/generator updates.

2. VAE (35 Points)

(a) Warmup

Training and test loss curves, along with generated images, are provided HERE.

(b) Effect of Interpolation in Latent Space

Interpolation Grid can be found HERE.

Observations in VAE

1. **Continuity**: The latent space of a VAE is encouraged to be continuous. This means that if you take two points that decode to meaningful images, the path between them is also likely to decode to something meaningful.
2. **Smooth Transitions**: As λ moves from 0 to 1, the digit "morphs" incrementally. For example, a "0" might slowly narrow and tilt until it resembles a "1". This is because the VAE maps the input space to a probability distribution (usually Gaussian) in the latent space.

3. **Completeness**: The KL-divergence term in the VAE loss function: $D_{KL}(q(z|x) \parallel p(z))$ acts as a regularizer, forcing the encoder to distribute latent vectors around the origin and preventing "holes" in the latent space.

Comparison with Non-Variational Autoencoders (AE)

A standard AE does not enforce latent structure:

- **Discontinuity**: The latent space is not regularized. The model only learns to map specific inputs to specific points to minimize reconstruction error.
- **Overfitting**: The encoder can "memorize" inputs by mapping them to isolated points in space.
- **Sparse Representations**: If you pick an intermediate point $z_\lambda = (1-\lambda)z_1 + \lambda z_2$ that wasn't used during training, the decoder has no obligation to produce a meaningful image.
- **Visual Result**: In an AE, an interpolation between a "0" and a "1" often looks like the "0" fading out while the "1" fades in (a simple overlay/blending), or it might pass through regions of "noise" where the output is unrecognizable.

(c) Convolutional VAE (Conv-VAE)

Implementation is provided in:

- main_conv.py
- interpolate_conv.py

Results can be found HERE.

Observation:
Compared to the fully connected VAE, the convolutional VAE produces:

- **sharper and clearer reconstructions**
- **better preservation of digit structure** (strokes, edges)
- **smoother and more realistic interpolations** in latent space
- faster convergence and slightly lower test loss in many cases

Advantage of Convolution over Fully Connected Layers

Convolutional layers are better suited for image data because they:

- **Preserve spatial structure**: Pixels that are close in the image remain close in feature maps, unlike FC layers which flatten the input.
- **Capture local patterns**: Filters learn meaningful features like edges, curves, and strokes.
- **Use parameter sharing**: Same filter is applied across the image → fewer parameters and better generalization.
- **Are more efficient**: Require fewer parameters than FC layers, reducing overfitting and improving training stability.

3. DDPM (50 Points)

(a) Warmup

The DDPM model was trained on MNIST using the provided implementation.

Observations

- Training and test loss decrease steadily over epochs
- Generated samples gradually improve from noise to digit-like structures
- Early epochs produce noisy patterns
- Later epochs show recognizable strokes and shapes

Generated samples and loss plots are provided HERE.

(b) DDIM Sampling

The original DDPM sampling process requires iterating through all timesteps (e.g., 1000 steps), making it computationally expensive.

Modification

A new method ddim_sampling() was implemented to:

- use a reduced set of timesteps (e.g., 20 steps)
- apply the DDIM update rule
- optionally use deterministic sampling (\eta = 0)

Observations

- Sampling is significantly faster (≈50× reduction in steps)
- Generated images remain meaningful and structurally similar
- Slight reduction in quality may occur with very few steps

Conclusion

DDIM improves efficiency by reducing sampling steps while maintaining reasonable image quality.

Generated DDIM samples are provided HERE.

(c) Conditioning on Class Label

The model was modified to incorporate class label information.

Implementation

- Added **label embedding** in the U-Net
- Combined label embedding with timestep embedding
- Modified forward pass: $\epsilon_\theta(x_t, t, y)$
- Updated training, evaluation, and sampling functions to include labels

Results

Generated samples are organized per class:

- each row corresponds to a digit (0–9)
- samples within a row resemble the target digit

Results are provided HERE. The code was also stored in model_partC.py and train_mnist_partC.py.

Observations

- The model successfully generates class-specific digits
- Conditioning enables **controlled generation**
- Sample quality improves with training

Conclusion

Class conditioning allows the diffusion model to generate specific digit classes on demand, making it more flexible and controllable compared to the unconditional model.